



Univerzitet u Nišu
Elektronski Fakultet



Serializable Snapshot Isolation mehanizam u PostgreSQL sistemu baza podataka

Mentor:
doc. dr Aleksandar Stanimirović

Student:
Filip Stamenković 2027

Niš, 2026.

Sadržaj

1. Uvod	3
2. Transakcije i ACID svojstva	4
2.1 ACID svojstva	4
3. Problemi konkurentnog izvršavanja transakcija i anomalije	5
3.1 Dirty Read	6
3.2 Non-Repeatable Read	6
3.3 Phantom Read	7
3.4 Lost Update	8
3.5 Write Skew anomalija	9
4. Nivoi izolacije transakcija i njihova implementacija u PostgreSQL-u	10
4.1 READ UNCOMMITTED	10
4.2 READ COMMITTED	10
4.3 REPEATABLE READ	11
4.4 SERIALIZABLE	12
4.5 Poređenje nivoa izolacije i PostgreSQL pristup izolaciji	12
5. Multi-Version Concurrency Control (MVCC) i Snapshot Isolation u PostgreSQL-u	13
5.1 Osnovni princip MVCC mehanizma	13
5.2 Transakcioni identifikatori	14
5.3 Snapshot koncept	15
5.4 Snapshot Isolation	15
5.5 VACUUM mehanizam	17
6. Serializable Snapshot Isolation (SSI) mehanizam u PostgreSQL-u	17
6.1 Šta je serializabilnost?	17
6.2 Ograničenja Snapshot Isolation modela	18
6.3 Praćenje zavisnosti između transakcija	19
6.4 Serialization Graph	19
6.5 Dangerous Structures	20
6.6 Predicate Locking	20
6.7 Serialization Failure	21
6.8 Prednosti i nedostaci Serializable Snapshot Isolation mehanizma	21
7. Praktična implementacija i demonstracija SSI mehanizma u PostgreSQL-u	22
7.1 Eksperimentalno okruženje	22
7.2 Write Skew anomalija korišćenjem REPEATABLE READ nivoa izolacije	24
7.3 Write Skew anomalija korišćenjem SERIALIZABLE nivoa izolacije	25
8. Zaključak	27
7. Literatura	28

1. Uvod

Savremeni sistemi baza podataka predstavljaju osnovu velikog broja informacionih sistema, poslovnih aplikacija i distribuiranih servisa koji zahtevaju pouzdanu i efikasnu obradu velike količine podataka. U modernim aplikacijama često postoji veliki broj istovremenih korisnika i procesa koji paralelno pristupaju istim podacima, zbog čega konkurentni pristup predstavlja jedan od bitnih izazova u projektovanju sistema baza podataka.

Prilikom izvršavanja više transakcija u isto vreme može doći do različitih problema konzistentnosti podataka, kao što su izgubljena ažuriranja, nekonzistentna čitanja ili narušavanje poslovnih pravila sistema. Kako bi se obezbedilo pravilno izvršavanje konkurentnih transakcija, sistemi baza podataka implementiraju različite mehanizme izolacije transakcija i kontrole konkurentnog pristupa podacima.

Tradicionalni pristupi kontroli konkurentnosti zasnivaju se na različitim tehnikama zaključavanja podataka, međutim savremeni sistemi često koriste naprednije mehanizme poput Multi-Version Concurrency Control (MVCC) pristupa. PostgreSQL predstavlja jedan od najpoznatijih relacionih sistema baza podataka koji koristi MVCC arhitekturu kako bi omogućio visok stepen konkurentnosti i efikasno izvršavanje transakcija.

Jedan od značajnih izazova kod konkurentnog izvršavanja transakcija jeste postizanje serializabilnosti, odnosno ponašanja sistema koje je ekvivalentno sekvencijalnom izvršavanju transakcija. Klasični mehanizmi za obezbeđivanje serializabilnosti često imaju značajan uticaj na performanse sistema zbog intenzivnog zaključavanja podataka i povećanog nivoa blokiranja transakcija.

Kako bi se prevazišli navedeni problemi, PostgreSQL implementira Serializable Snapshot Isolation (SSI) mehanizam koji kombinuje prednosti Snapshot Isolation pristupa i serializabilne izolacije transakcija. SSI omogućava visok nivo konkurentnosti uz istovremeno očuvanje konzistentnosti podataka i detekciju potencijalno opasnih konflikata između transakcija.

Cilj ovog seminarskog rada jeste analiza Serializable Snapshot Isolation mehanizma u PostgreSQL sistemu baza podataka, sa posebnim fokusom na način funkcionisanja konkurentnih transakcija, detekciju anomalija i praktične primere rada sistema u različitim scenarijima konkurentnog pristupa podacima. Pored teorijske analize, rad obuhvata i praktične demonstracije konkurentnih transakcija i mehanizama koje PostgreSQL koristi za očuvanje konzistentnosti podataka.

2. Transakcije i ACID svojstva

Transakcija predstavlja osnovnu jedinicu rada u sistemima baza podataka. Može se posmatrati kao logička celina koja se sastoji od jedne ili više operacija nad podacima, pri čemu se sve operacije moraju izvršiti uspešno ili nijedna od njih ne sme ostaviti trajne posledice u bazi podataka. Na taj način se čuva integritet i konzistentnost podataka čak i u slučaju greške, prekida rada sistema ili konkurentnog izvršavanja više transakcija.

Primer transakcije može biti prenos novca između dva bankovna računa. Takva operacija podrazumeva umanjeње stanja na jednom računu i uvećanje stanja na drugom računu. Ukoliko bi se izvršila samo jedna od ove dve operacije, sistem bi ostao u nekonzistentnom stanju. Zbog toga se obe operacije izvršavaju u okviru jedne transakcije.

Pouzdan rad transakcija zasniva se na ACID principima, koji predstavljaju skup osobina koje transakcija mora da zadovolji kako bi se garantovala ispravnost rada sistema.

2.1 ACID svojstva

ACID principi predstavljaju osnovu pouzdanog rada relacionih sistema baza podataka. Oni omogućavaju bezbedno izvršavanje velikog broja konkurentnih transakcija uz očuvanje integriteta podataka i predvidivo ponašanje sistema.

ACID akronim čine 4 reči tj. principa: Atomicity, Consistency, Isolation, Durability.

Atomicity (atomičnost) predstavlja osobinu prema kojoj se transakcija izvršava kao nedeljiva celina. Sve operacije koje pripadaju transakciji moraju biti uspešno izvršene kako bi promene postale trajne. Ukoliko dođe do greške tokom izvršavanja, sistem poništava sve prethodno izvršene promene korišćenjem rollback operacije.

Na primer, prilikom prenosa novca između računa nije dozvoljeno da sredstva budu skinuta sa jednog računa, a da ne budu uplaćena na drugi. U slučaju greške, sistem mora vratiti bazu podataka u prethodno stanje [5][6].

Consistency (konzistentnost) podrazumeva da svaka transakcija prevodi bazu podataka iz jednog validnog stanja u drugo validno stanje. Tokom izvršavanja transakcije moraju biti poštovana sva definisana pravila integriteta, ograničenja i poslovna pravila sistema.

Primer konzistentnosti može biti ograničenje koje zahteva da stanje bankovnog računa ne sme biti negativno. Nakon izvršavanja transakcije baza mora ostati u stanju koje zadovoljava ovo pravilo.

Važno je naglasiti da je odgovornost za konzistentnost podeljena između sistema baze podataka i same aplikacije, koja definiše poslovna pravila koja podaci moraju zadovoljiti [5][6].

Isolation (izolacija) predstavlja osobinu koja omogućava da se konkurentne transakcije izvršavaju nezavisno jedna od druge. Svaka transakcija treba da se ponaša kao da je jedina aktivna transakcija u sistemu, bez obzira na prisustvo drugih istovremenih operacija.

U praksi potpuno izolovano izvršavanje može značajno smanjiti performanse sistema, zbog čega moderni DBMS sistemi implementiraju različite nivoe izolacije koji predstavljaju kompromis između konzistentnosti i performansi.

Problemi koji mogu nastati usled nedovoljne izolacije uključuju pojave kao što su dirty read, non-repeatable read, phantom read i write skew anomalije [5][6][7].

Durability (trajnost) garantuje da će sve promene potvrđene uspešno završenom transakcijom ostati sačuvane čak i u slučaju nestanka električne energije, pada operativnog sistema ili kvara hardvera.

Savremeni sistemi baza podataka ostvaruju trajnost korišćenjem različitih mehanizama za evidentiranje promena, kao što su Write-Ahead Logging (WAL) i checkpoint mehanizmi. Nakon potvrde transakcije, sistem garantuje da će njeni rezultati biti sačuvani i moći da se obnove tokom procesa oporavka sistema [5][6].

Posebno važno svojstvo za ovaj rad je svojstvo izolacije, koje direktno utiče na način na koji sistem upravlja konkurentnim pristupom podacima. Razvoj različitih nivoa izolacije i naprednih mehanizama poput Snapshot Isolation i Serializable Snapshot Isolation predstavlja pokušaj postizanja ravnoteže između visokih performansi i očuvanja konzistentnosti podataka u savremenim bazama podataka.

3. Problemi konkurentnog izvršavanja transakcija i anomalije

Moderni sistemi baza podataka projektovani su tako da podržavaju istovremeni rad velikog broja korisnika i aplikacija. U takvom okruženju više transakcija može paralelno pristupati istim podacima, čitati ih ili menjati njihove vrednosti. Iako konkurentno izvršavanje značajno povećava iskorišćenost resursa i performanse sistema, ono istovremeno može dovesti do različitih problema konzistentnosti podataka.

Ukoliko sistem ne obezbedi odgovarajuću izolaciju između transakcija, rezultat njihovog paralelnog izvršavanja može se razlikovati od rezultata koji bi bio dobijen njihovim sekvencijalnim izvršavanjem. Takve situacije nazivaju se anomalijama konkurentnog izvršavanja transakcija.

3.1 Dirty Read

Dirty Read predstavlja situaciju u kojoj jedna transakcija čita podatke koje je druga transakcija izmenila, ali još uvek nije potvrdila izvršavanje korišćenjem COMMIT operacije.

Posmatrajmo sledeći primer:

Transakcija T1

```
BEGIN;  
UPDATE racun  
SET stanje = stanje - 1000  
WHERE id = 1;
```

Transakcija T2

```
SELECT stanje  
FROM racun  
WHERE id = 1;
```

Ukoliko T2 pročitava novu vrednost pre nego što T1 izvrši COMMIT, a T1 kasnije izvrši ROLLBACK, T2 je koristila podatke koji nikada nisu postali deo validnog stanja baze podataka.

Zbog mogućnosti nastanka problema konzistentnosti, PostgreSQL ne dozvoljava Dirty Read ni na najnižem podržanom nivou izolacije [5][6].

3.2 Non-Repeatable Read

Non-Repeatable Read nastaje kada transakcija dva puta pročita isti red podataka, a između ta dva čitanja druga transakcija izmeni i potvrdi promenu nad tim redom.

Primer:

Transakcija T1

```
BEGIN;  
SELECT stanje  
FROM racun  
WHERE id = 1;
```

Rezultat: 5000

U međuvremenu:

Transakcija T2

```
UPDATE racun  
SET stanje = 6000  
WHERE id = 1;  
COMMIT;
```

T1 zatim ponovo izvršava isto čitanje:

```
SELECT stanje  
FROM racun  
WHERE id = 1;
```

Rezultat: 6000

Iako je T1 u okviru iste transakcije dva puta pročitala isti podatak, dobila je različite rezultate. Ovakvo ponašanje može predstavljati problem za aplikacije koje očekuju stabilan pogled na podatke tokom celog trajanja transakcije [5][7].

3.3 Phantom Read

Phantom Read nastaje kada transakcija ponovo izvrši upit nad skupom redova, a između dva izvršavanja druga transakcija ubaci ili obriše redove koji zadovoljavaju uslov pretrage.

Primer:

Transakcija T1

```
SELECT *  
FROM zaposleni  
WHERE plata > 100000;
```

Rezultat: 5 redova

U međuvremenu:

Transakcija T2

```
INSERT INTO zaposleni  
VALUES (...);
```

```
COMMIT;
```

Nakon toga T1 ponovo izvršava isti upit:

```
SELECT *  
FROM zaposleni  
WHERE plata > 100000;
```

Rezultat: 6 redova

Novi red koji se pojavio naziva se fantomski red (phantom row), dok se sama pojava naziva Phantom Read [5][7].

3.4 Lost Update

Lost Update predstavlja situaciju u kojoj dve transakcije istovremeno čitaju istu vrednost i na osnovu nje vrše izmene, pri čemu jedna promena prepisuje drugu.

Pretpostavimo da stanje računa iznosi 1000.

Transakcija T1

```
SELECT stanje FROM racun WHERE id = 1;
```

Rezultat: 1000

Transakcija T2

```
SELECT stanje FROM racun WHERE id = 1;
```

Rezultat: 1000

T1 povećava stanje za 100:

```
UPDATE racun  
SET stanje = 1100  
WHERE id = 1;
```

T2 povećava stanje za 200:

```
UPDATE racun  
SET stanje = 1200
```



```
WHERE id = 1;
```

Konačno stanje postaje 1200, dok je očekivani rezultat 1300. Promena koju je izvršila T1 praktično je izgubljena.

3.5 Write Skew anomalija

Write Skew predstavlja jednu od najznačajnijih anomalija, jer upravo ona predstavlja motivaciju za razvoj Serializable Snapshot Isolation mehanizma.

Posmatrajmo sistem za raspored dežurstava u bolnici. Pravilo sistema nalaže da najmanje jedan lekar mora biti na dežurstvu u svakom trenutku.

Početno stanje:

Tabela 1: Početno stanje dežurstva.

Lekar	Na dežurstvu
Ana	DA
Marko	DA

Dve transakcije se izvršavaju paralelno. Prva transakcija proverava da li je Marko na dežurstvu i, pošto jeste, uklanja Anu sa dežurstva. Druga transakcija proverava da li je Ana na dežurstvu i, pošto jeste, uklanja Marka sa dežurstva.

Obe transakcije pojedinačno poštuju poslovno pravilo, međutim nakon njihovog izvršavanja stanje sistema postaje:

Tabela 2: Stanje dežurstva nakon izvršavanja transakcija.

Lekar	Na dežurstvu
Ana	NE
Marko	NE

Poslovno pravilo je narušeno, iako nijedna pojedinačna transakcija nije izvršila neispravnu operaciju.

Write Skew anomalija predstavlja jedan od najpoznatijih problema Snapshot Isolation pristupa i upravo njeno sprečavanje predstavlja osnovni cilj Serializable Snapshot Isolation mehanizma [3][8].

Opisane anomalije pokazuju da konkurentno izvršavanje transakcija može dovesti do različitih oblika nekonzistentnosti podataka. Zbog toga sistemi baza podataka implementiraju različite nivo izolacije koji definišu koje anomalije su dozvoljene, a koje moraju biti sprečene.

4. Nivoi izolacije transakcija i njihova implementacija u PostgreSQL-u

Kako bi se kontrolisale anomalije nastale usled konkurentnog izvršavanja transakcija, sistemi baza podataka implementiraju različite nivo izolacije. Nivo izolacije definiše koliko su transakcije međusobno izolovane i koje vrste anomalija mogu da se pojave tokom njihovog izvršavanja.

SQL standard definiše četiri osnovna nivoa izolacije transakcija: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ i SERIALIZABLE. Svaki naredni nivo pruža viši stepen zaštite od anomalija, ali često uz cenu smanjenih performansi i manje konkurentnosti sistema [5][6].

PostgreSQL implementira sopstveni pristup izolaciji zasnovan na Multi-Version Concurrency Control (MVCC) mehanizmu, zbog čega se njegovo ponašanje u određenim slučajevima razlikuje od klasičnih implementacija opisanih u SQL standardu.

4.1 READ UNCOMMITTED

READ UNCOMMITTED predstavlja najniži nivo izolacije definisan SQL standardom. Na ovom nivou transakcije mogu se čitati izmene koje druge transakcije još uvek nisu potvrdile izvršavanjem COMMIT operacije.

Posledica ovakvog ponašanja jeste mogućnost pojave Dirty Read anomalije.

Međutim, PostgreSQL ne podržava stvarni READ UNCOMMITTED nivo izolacije. Zbog MVCC arhitekture, svaki pokušaj korišćenja READ UNCOMMITTED nivoa automatski se tretira kao READ COMMITTED [1].

Na taj način PostgreSQL eliminiše mogućnost Dirty Read anomalije bez dodatnih mehanizama zaključavanja.

4.2 READ COMMITTED

READ COMMITTED predstavlja podrazumevani nivo izolacije u PostgreSQL sistemu [1].

Kod ovog nivoa svaka SQL naredba dobija sopstveni snapshot baze podataka koji sadrži samo podatke potvrđene pre početka izvršavanja te naredbe.

Primer:

```
BEGIN;  
  
SELECT stanje  
FROM racun  
WHERE id = 1;
```

Ukoliko druga transakcija izvrši:

```
UPDATE racun  
SET stanje = 6000  
WHERE id = 1;  
  
COMMIT;
```

Sledeće izvršavanje istog SELECT upita unutar prve transakcije može vratiti novu vrednost.

Zbog toga su na ovom nivou moguće sledeće anomalije:

- Non-Repeatable Read
- Phantom Read
- Write Skew

Prednosti READ COMMITTED nivoa su visoka konkurentnost, mali broj konflikata i dobre performanse.

4.3 REPEATABLE READ

REPEATABLE READ u PostgreSQL-u koristi Snapshot Isolation pristup [1][2]. Za razliku od READ COMMITTED nivoa, kompletna transakcija koristi isti snapshot baze podataka tokom celog svog trajanja. Prilikom pokretanja transakcije kreira se konzistentan pogled na bazu podataka koji ostaje nepromenjen do završetka transakcije.

Primer:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Svi naredni SELECT upiti vide identično stanje baze bez obzira na izmene koje druge transakcije potvrde tokom njenog trajanja.

Na ovaj način eliminišu se:

- Dirty Read
- Non-Repeatable Read
- Phantom Read

Međutim, Snapshot Isolation i dalje ne garantuje punu serializabilnost. U određenim situacijama moguće su anomalije kao što je Write Skew. Zbog toga se REPEATABLE READ nivo često posmatra kao kompromis između performansi i konzistentnosti.

4.4 SERIALIZABLE

SERIALIZABLE predstavlja najviši nivo izolacije dostupan u PostgreSQL sistemu. Cilj ovog nivoa jeste da garantuje da rezultat konkurentnog izvršavanja transakcija bude identičan rezultatu nekog mogućeg sekvencijalnog izvršavanja tih istih transakcija.

Tradicionalni DBMS sistemi često ostvaruju serializabilnost korišćenjem intenzivnog zaključavanja podataka. Takav pristup može dovesti do smanjenja performansi i povećanog broja blokiranih transakcija.

PostgreSQL koristi drugačiji pristup poznat kao Serializable Snapshot Isolation (SSI) [1][3]. Umesto agresivnog zaključavanja, SSI prati zavisnosti između transakcija i detektuje potencijalno opasne obrasce koji bi mogli dovesti do narušavanja serializabilnosti. Kada sistem detektuje takvu situaciju, jedna od transakcija se prekida porukom:

ERROR: could not serialize access due to read/write dependencies among transactions

Aplikacija nakon toga može ponoviti izvršavanje transakcije.

Na ovaj način PostgreSQL uspeva da obezbedi visok nivo konzistentnosti uz značajno bolju konkurentnost u odnosu na tradicionalne pristupe bazirane na zaključavanju.

4.5 Poređenje nivoa izolacije i PostgreSQL pristup izolaciji

Tabela 3: Mogućnost pojavljivanja pojedinih anomalija kod različitih nivoa izolacije.

Anomalija	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Dirty Read	Ne	Ne	Ne
Non-Repeatable Read	Da	Ne	Ne
Phantom Read	Da	Ne	Ne
Write Skew	Da	Da	Ne

Može se primetiti da SERIALIZABLE nivo pruža najviši stepen zaštite od anomalija, dok READ COMMITTED nudi najbolje performanse i najveću konkurentnost.

Jedna od ključnih karakteristika PostgreSQL sistema jeste oslanjanje na MVCC arhitekturu umesto na intenzivno zaključavanje podataka. Zahvaljujući tome veliki broj transakcija može istovremeno pristupati podacima bez međusobnog blokiranja.

MVCC omogućava kreiranje više verzija istog reda podataka, pri čemu svaka transakcija vidi verziju koja odgovara njenom snapshot-u. Ovakav pristup predstavlja osnovu za implementaciju Snapshot Isolation i Serializable Snapshot Isolation mehanizama.

5. Multi-Version Concurrency Control (MVCC) i Snapshot Isolation u PostgreSQL-u

Efikasna kontrola konkurentnog pristupa podacima predstavlja jedan od osnovnih zahteva savremenih sistema baza podataka. Tradicionalni pristupi zasnovani na zaključavanju često dovode do blokiranja transakcija i smanjenja ukupnih performansi sistema. Kako bi se povećao stepen konkurentnosti, PostgreSQL koristi Multi-Version Concurrency Control (MVCC) mehanizam [2].

MVCC omogućava istovremeno izvršavanje velikog broja transakcija bez potrebe za intenzivnim zaključavanjem podataka. Umesto da transakcije međusobno blokiraju pristup redovima, sistem održava više verzija istog podatka i svakoj transakciji prikazuje odgovarajuću verziju u skladu sa njenim pogledom na bazu podataka. Ovakav pristup predstavlja osnovu za implementaciju Snapshot Isolation i Serializable Snapshot Isolation mehanizama u PostgreSQL sistemu.

5.1 Osnovni princip MVCC mehanizma

Kod klasičnih sistema zasnovanih na zaključavanju, transakcija koja želi da izmeni podatak zaključava odgovarajući red dok se operacija ne završi. Druge transakcije tada moraju da čekaju oslobađanje zaključavanja pre nego što mogu pristupiti istom podatku. MVCC koristi drugačiji pristup. Kada se red izmeni, PostgreSQL ne prepisuje postojeću vrednost, već kreira novu verziju reda. Stara verzija ostaje dostupna drugim transakcijama koje su započele pre izvršene izmene.

Na taj način više transakcija može istovremeno pristupati istim podacima bez međusobnog blokiranja.

Na primer, pretpostavimo da red sadrži sledeću vrednost:

Tabela 4: Početna vrednost kolone stanje.

ID	Stanje
1	1000

Transakcija T1 izvršava:

```
UPDATE racun  
SET stanje = 1200  
WHERE id = 1;
```

PostgreSQL ne menja postojeći red direktno, već kreira novu verziju:

Tabela 5: Novokreirana verzija reda i stara verzija.

Verzija	Stanje
Stara	1000
Nova	1200

Transakcije koje su započele pre izmene i dalje vide vrednost 1000, dok novije transakcije vide vrednost 1200.

5.2 Transakcioni identifikatori

Svaka transakcija u PostgreSQL sistemu dobija jedinstveni Transaction ID (XID). Prilikom kreiranja ili izmene reda PostgreSQL evidentira dva važna atributa:

- xmin
- xmax

Atribut xmin sadrži identifikator transakcije koja je kreirala trenutnu verziju reda. Atribut xmax sadrži identifikator transakcije koja je obrisala ili zamenila tu verziju reda.

Vizuelno, red može izgledati ovako:

Tabela 6: Primer vrednosti transakcionih atributa.

Kolona	Vrednost
xmin	100

xmax	150
stanje	1000

To znači da je red kreirala transakcija 100, a da je njegova verzija postala nevažeća nakon izvršavanja transakcije 150. Prilikom svakog čitanja PostgreSQL koristi ove informacije kako bi odredio da li određena verzija reda treba da bude vidljiva konkretnoj transakciji [2].

5.3 Snapshot koncept

Ključna komponenta MVCC arhitekture jeste snapshot. Snapshot predstavlja logički prikaz baze podataka u određenom trenutku. Kada transakcija započne izvršavanje, PostgreSQL formira skup informacija o aktivnim i završenim transakcijama. Na osnovu tih informacija određuje se koje verzije redova su vidljive toj transakciji. Svaka transakcija koristi svoj snapshot kako bi obezbedila konzistentan pogled na podatke.

Zahvaljujući snapshot mehanizmu:

- transakcije mogu paralelno čitati podatke,
- čitanja ne blokiraju pisanja,
- pisanja uglavnom ne blokiraju čitanja,
- povećava se konkurentnost sistema.

Ovo predstavlja jednu od najvećih prednosti PostgreSQL arhitekture u odnosu na sisteme koji se dominantno oslanjaju na zaključavanje.

5.4 Snapshot Isolation

Snapshot Isolation predstavlja model izolacije zasnovan na MVCC mehanizmu [2][4]. Kod Snapshot Isolation pristupa transakcija koristi konzistentan snapshot baze podataka tokom celog svog trajanja. To znači da svi SELECT upiti unutar iste transakcije vide identično stanje baze, bez obzira na izmene koje druge transakcije izvrše nakon njenog početka.

Primer:

Transakcija T1:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

T1 čita stanje računa:

```
SELECT stanje
FROM racun
```

```
WHERE id = 1;
```

Rezultat: 1000

Druga transakcija:

```
UPDATE racun  
SET stanje = 2000  
WHERE id = 1;
```

```
COMMIT;
```

Kada T1 ponovo pročitava isti podatak:

```
SELECT stanje  
FROM racun  
WHERE id = 1;
```

rezultat će i dalje biti: 1000

Transakcija nastavlja da koristi originalni snapshot sve do svog završetka.

Snapshot Isolation pruža značajne prednosti u odnosu na tradicionalne modele konkurentnosti, neke od njih su:

- visok nivo konkurentnosti,
- eliminacija Dirty Read anomalije,
- eliminacija Non-Repeatable Read anomalije,
- smanjen broj blokiranja transakcija,
- veća iskorišćenost procesorskih resursa.

Iako Snapshot Isolation eliminiše veliki broj problema konkurentnog izvršavanja, on ne garantuje potpunu serializabilnost. Određene anomalije i dalje mogu nastati, pri čemu je najpoznatija Write Skew anomalija [8].

Posmatrajmo primer dežurnih lekara opisan u prethodnom poglavlju. Dve transakcije koriste isti snapshot baze podataka i zaključuju da drugi lekar ostaje na dežurstvu. Obe transakcije zatim uklanjaju različite lekare sa dežurstva. Pošto ne dolazi do direktnog konflikta nad istim redom, obe transakcije mogu uspešno da se potvrde. Konačni rezultat predstavlja narušavanje poslovnog pravila sistema. Takve situacije pokazuju da Snapshot Isolation nije dovoljan za aplikacije koje zahtevaju najviši nivo konzistentnosti podataka.

MVCC i Snapshot Isolation predstavljaju osnovu na kojoj PostgreSQL implementira Serializable Snapshot Isolation mehanizam. SSI koristi sve prednosti Snapshot Isolation pristupa, uključujući

visok stepen konkurentnosti i minimalno blokiranje transakcija. Međutim, dodatno prati zavisnosti između transakcija kako bi detektovao potencijalne situacije koje mogu dovesti do narušavanja serializabilnosti.

5.5 VACUUM mehanizam

Pošto MVCC kontinuirano generiše nove verzije redova, tokom vremena u bazi nastaje veliki broj zastarelih verzija podataka. Ukoliko se one ne uklanjaju, dolazi do povećanja zauzeća prostora i smanjenja performansi sistema. Zbog toga PostgreSQL koristi VACUUM mehanizam.

Zadatak VACUUM procesa jeste:

- uklanjanje zastarelih verzija redova,
- oslobađanje prostora,
- održavanje performansi sistema,
- sprečavanje problema sa Transaction ID brojačima.

Savremene PostgreSQL instalacije uglavnom koriste Autovacuum servis koji automatski pokreće procese čišćenja kada je to potrebno.

6. Serializable Snapshot Isolation (SSI) mehanizam u PostgreSQL-u

Serializable Snapshot Isolation (SSI) predstavlja mehanizam koji PostgreSQL koristi za implementaciju SERIALIZABLE nivoa izolacije transakcija. Cilj SSI pristupa jeste da obezbedi serializabilnost transakcija uz zadržavanje visokog nivoa konkurentnosti karakterističnog za Snapshot Isolation model.

Za razliku od tradicionalnih sistema koji serializabilnost ostvaruju intenzivnim zaključavanjem podataka, PostgreSQL koristi pristup zasnovan na analizi zavisnosti između transakcija. Time se smanjuje broj blokiranja i omogućava efikasnije iskorišćenje sistemskih resursa.

SSI je prvi put uveden u PostgreSQL verziji 9.1 i predstavlja jednu od najznačajnijih karakteristika sistema za obradu konkurentnih transakcija.

6.1 Šta je serializabilnost?

Serializabilnost predstavlja najviši nivo izolacije transakcija. Sistem je serializabilan ukoliko rezultat paralelnog izvršavanja više transakcija odgovara rezultatu nekog mogućeg sekvencijalnog izvršavanja tih istih transakcija [5][7].

Na primer, ukoliko se izvršavaju dve transakcije:

- T1
- T2

Serializabilnost garantuje da će krajnji rezultat biti identičan jednom od sledećih scenarija:

$T1 \rightarrow T2$ ili $T2 \rightarrow T1$

Bez obzira na to što se transakcije izvršavaju paralelno. Na taj način eliminišu se anomalije koje mogu dovesti do nekonzistentnosti podataka i narušavanja poslovnih pravila sistema.

6.2 Ograničenja Snapshot Isolation modela

Kao što je prikazano u prethodnom poglavlju, Snapshot Isolation eliminiše veliki broj anomalija, ali ne garantuje potpunu serializabilnost.

Najpoznatiji primer predstavlja Write Skew anomalija.

Posmatrajmo tabelu:

Tabela 7: Stanje tabele dežurstva pre transakcija T1 i T2.

Lekar	Na dežurstvu
Ana	DA
Marko	DA

Transakcija T1:

- proverava da li je Marko na dežurstvu,
- ukoliko jeste, uklanja Anu sa dežurstva.

Transakcija T2:

- proverava da li je Ana na dežurstvu,
- ukoliko jeste, uklanja Marka sa dežurstva.

Obe transakcije koriste isti snapshot baze podataka i obe zaključuju da poslovno pravilo ostaje ispunjeno.

Nakon izvršavanja:

Tabela 8: Stanje tabele dežurstva nakon transakcija T1 i T2.

Lekar	Na dežurstvu
Ana	NE
Marko	NE

Poslovno pravilo je narušeno, iako nijedna pojedinačna transakcija nije izvršila neispravnu operaciju. Ovakve situacije predstavljaju osnovni motiv za razvoj SSI mehanizma.

6.3 Praćenje zavisnosti između transakcija

Osnovna ideja SSI pristupa jeste praćenje zavisnosti između transakcija tokom njihovog izvršavanja.

PostgreSQL analizira:

- operacije čitanja,
- operacije pisanja,
- redosled izvršavanja transakcija.

Posebna pažnja posvećuje se read-write (rw) konfliktima.

RW konflikt nastaje kada:

1. jedna transakcija pročita određeni podatak,
2. druga transakcija kasnije izmeni isti podatak.

Takva situacija može predstavljati potencijalni izvor anomalija [3].

Tokom izvršavanja PostgreSQL evidentira ove konflikte i koristi ih za izgradnju logičkog modela zavisnosti između transakcija.

6.4 Serialization Graph

Za analizu serializabilnosti koristi se koncept Serialization Graph-a. Čvorovi grafa predstavljaju transakcije. Grane predstavljaju zavisnosti između njih.

Primer:

$T1 \rightarrow T2$

znači da rezultat transakcije T2 zavisi od aktivnosti transakcije T1.

Ukoliko graf ne sadrži cikluse, izvršavanje je serializabilno.

Međutim, ukoliko se formira ciklus: $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

nije moguće pronaći sekvencijalni redosled izvršavanja koji odgovara stvarnom ponašanju sistema. Takva situacija predstavlja potencijalno narušavanje serializabilnosti [3][8].

6.5 Dangerous Structures

PostgreSQL ne konstruiše kompletan serialization graph tokom rada sistema jer bi takav pristup bio preskup sa aspekta performansi. Umesto toga koristi koncept dangerous structures [3].

Dangerous structure predstavlja obrazac konflikata koji može dovesti do nastanka ciklusa u serialization graph-u.

Najčešći oblik izgleda ovako:

$T1 \rightarrow_{rw} T2 \rightarrow_{rw} T3$

Ukoliko se ispune dodatni uslovi, PostgreSQL zaključuje da postoji mogućnost narušavanja serializabilnosti. U tom trenutku jedna od uključenih transakcija biva prekinuta.

Ovakav pristup omogućava značajno manji režijski trošak u poređenju sa analizom kompletnog grafa zavisnosti.

6.6 Predicate Locking

Jedan od izazova SSI mehanizma jeste detekcija Phantom Read anomalija.

Posmatrajmo upit:

```
SELECT *  
FROM zaposleni  
WHERE plata > 100000;
```

U međuvremenu druga transakcija može dodati nov red koji zadovoljava isti uslov. Kako bi otkrio ovakve situacije, PostgreSQL koristi koncept poznat kao Predicate Locking [1][3][9].

Važno je napomenuti da se ne radi o klasičnim zaključavanjima koja blokiraju pristup podacima. Umesto toga koriste se tzv. SIREAD zaključavanja koja služe isključivo za praćenje zavisnosti između transakcija. Njihova svrha nije blokiranje već evidencija potencijalnih konflikata.

Zahvaljujući tome moguće je detektovati anomalije bez značajnog smanjenja konkurentnosti sistema.

6.7 Serialization Failure

Kada PostgreSQL detektuje opasnu strukturu koja može dovesti do narušavanja serializabilnosti, jedna od uključenih transakcija biva prekinuta.

Aplikacija dobija poruku:

ERROR: could not serialize access due to read/write dependencies among transactions

ili

SQLSTATE 40001

Ovakva greška ne predstavlja kvar sistema niti gubitak podataka već pokazuje da je SSI mehanizam uspešno sprečio potencijalnu anomaliju [1][3]. U praksi se preporučuje ponovno izvršavanje prekinute transakcije.

Pošto SERIALIZABLE nivo izolacije može rezultovati serialization failure greškama, aplikacije moraju biti spremne da ponove izvršavanje transakcije. Većina modernih aplikacija implementira automatski retry mehanizam upravo za ovakve situacije.

6.8 Prednosti i nedostaci Serializable Snapshot Isolation mehanizma

Glavne prednosti Serializable Snapshot Isolation mehanizma su:

- garantovana serializabilnost,
- visok nivo konkurentnosti,
- minimalno blokiranje transakcija,
- dobra skalabilnost sistema,
- eliminacija Write Skew anomalije,
- efikasniji rad u odnosu na tradicionalne lock-based pristupe.

Zahvaljujući ovim karakteristikama PostgreSQL uspeva da obezbedi najviši nivo konzistentnosti bez značajnog smanjenja performansi.

Iako SSI pruža brojne prednosti, postoje i određeni nedostaci:

- dodatna memorija za praćenje konflikata,
- povećan CPU overhead,
- mogućnost serialization failure grešaka,
- potreba za implementacijom retry logike u aplikacijama.

U sistemima sa veoma velikim brojem konkurentnih transakcija može doći do povećanog broja ponovnih izvršavanja transakcija, što može uticati na ukupne performanse sistema.

Međutim, u većini realnih scenarija ove mane su znatno manje od problema koji bi nastali usled narušavanja konzistentnosti podataka.

Serializable Snapshot Isolation predstavlja jedno od najznačajnijih unapređenja PostgreSQL sistema u oblasti kontrole konkurentnog pristupa podacima. Kombinovanjem MVCC arhitekture, Snapshot Isolation modela i analize konflikata između transakcija, PostgreSQL uspeva da obezbedi ponašanje ekvivalentno sekvencijalnom izvršavanju transakcija bez potrebe za agresivnim zaključavanjem podataka.

Ovakav pristup omogućava istovremeno postizanje visokog nivoa konzistentnosti i dobrih performansi, što SSI čini pogodnim za primenu u sistemima koji obrađuju veliki broj konkurentnih transakcija i zahtevaju stroge garancije ispravnosti podataka.

7. Praktična implementacija i demonstracija SSI mehanizma u PostgreSQL-u

U cilju praktične provere karakteristika Serializable Snapshot Isolation (SSI) mehanizma, sproveden je niz eksperimenata nad PostgreSQL sistemom baze podataka. Eksperimenti su osmišljeni tako da demonstriraju ponašanje konkurentnih transakcija pod različitim nivoima izolacije, sa fokusom na Write Skew anomaliju i način na koji PostgreSQL sprečava njeno nastajanje korišćenjem SSI mehanizma.

Praktični deo rada realizovan je korišćenjem dve nezavisne sesije baze podataka kako bi se simuliralo paralelno izvršavanje transakcija.

7.1 Eksperimentalno okruženje

Za potrebe eksperimenata korišćen je PostgreSQL sistem baze podataka pokrenut u Docker okruženju. Pokretanje baze izvršeno je pomoću Docker Compose alata gde je prethodno u konfiguracionom fajlu definisan skup karakteristika baze.

```
services:
  ssi-db:
    container_name: postgres_ssi
    image: postgres:latest
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
```

```

    PGDATA: /data/postgres
    POSTGRES_DB: bank
volumes:
  - db:/data/postgres
  - ./sql:/sql
ports:
  - "5332:5432"
networks:
  - mynet
restart: unless-stopped
healthcheck:
  test: [ "CMD-SHELL", "pg_isready -d postgres" ]
  interval: 30s
  timeout: 10s
  retries: 5

```

Listing 1: Docker Compose konfiguracija za demonstraciju SSI mehanizma.

Nakon pokretanja servera kreirana je tabela doctors koja simulira raspored dežurnih lekara:

```

DROP TABLE IF EXISTS doctors;

CREATE TABLE doctors (
  id INT PRIMARY KEY,
  name TEXT NOT NULL,
  on_call BOOLEAN NOT NULL
);

INSERT INTO doctors (id, name, on_call) VALUES
(1, 'Ana', TRUE),
(2, 'Marko', TRUE);

SELECT * FROM doctors ORDER BY id;

```

Tokom svih eksperimenata korišćeno je sledeće poslovno pravilo:

U svakom trenutku najmanje jedan lekar mora biti na dežurstvu.

Ovo pravilo predstavlja osnovu za demonstraciju Write Skew anomalije i omogućava jednostavnu proveru konzistentnosti sistema nakon izvršavanja konkurentnih transakcija.

7.2 Write Skew anomalija korišćenjem REPEATABLE READ nivoa izolacije

Prvi eksperiment izveden je korišćenjem REPEATABLE READ nivoa izolacije, koji u PostgreSQL sistemu implementira Snapshot Isolation model.

Otvorene su dve nezavisne psql sesije. U prvoj sesiji pokrenuta je sledeća transakcija:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
  
SELECT * FROM doctors WHERE on_call = TRUE ORDER BY id;  
  
UPDATE doctors  
SET on_call = FALSE  
WHERE id = 1;
```

Istovremeno je u drugoj sesiji izvršena sledeća transakcija:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
  
SELECT * FROM doctors WHERE on_call = TRUE ORDER BY id;  
  
UPDATE doctors  
SET on_call = FALSE  
WHERE id = 2;
```

Obe transakcije videle su identično stanje baze podataka u kojem su oba lekara bila na dežurstvu. Na osnovu toga svaka transakcija je zaključila da može bezbedno ukloniti jednog lekara sa dežurstva.

Nakon izvršavanja UPDATE operacija obe transakcije su uspešno potvrđene korišćenjem COMMIT naredbe.

Po završetku eksperimenta izvršen je sleći upit:

```
SELECT * FROM doctors ORDER BY id;
```

Rezultat je sledeći:

id	name	on_call
1	Ana	f
2	Marko	f

(2 rows)

Slika 1: Rezultat transakcija i pojava Write Skew anomalije na REPEATABLE READ nivou izolacije.

Dobijeni rezultat pokazuje da je poslovno pravilo sistema narušeno, pošto nijedan lekar više nije na dežurstvu.

Ovaj scenario predstavlja klasičan primer Write Skew anomalije. Iako su obe transakcije pojedinačno izvršene ispravno i na osnovu konzistentnog snapshot-a baze podataka, njihovo paralelno izvršavanje dovelo je do stanja koje nije moguće dobiti nijednim sekvencijalnim redosledom izvršavanja.

7.3 Write Skew anomalija korišćenjem SERIALIZABLE nivoa izolacije

Nakon demonstracije Write Skew anomalije izvršen je identičan eksperiment korišćenjem SERIALIZABLE nivoa izolacije.

Tabela doctors vraćena je u početno stanje:

```
UPDATE doctors
SET on_call = TRUE;

SELECT * FROM doctors ORDER BY id;
```

Prva sesija:

```
BEGIN ISOLATION LEVEL SERIALIZABLE;

SELECT * FROM doctors WHERE on_call = TRUE;

UPDATE doctors
SET on_call = FALSE
WHERE id = 1;
```

Druga sesija:

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

```
SELECT * FROM doctors WHERE on_call = TRUE;
```

```
UPDATE doctors  
SET on_call = FALSE  
WHERE id = 2;
```

Obe transakcije ponovo vide identično početno stanje baze podataka. Nakon izvršavanja UPDATE operacija potvrđena je prva transakcija:

```
COMMIT;
```

Međutim, prilikom potvrđivanja druge transakcije PostgreSQL prijavljuje sledeću grešku:

```
ERROR: could not serialize access due to read/write dependencies among transactions  
DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt.  
HINT: The transaction might succeed if retried.  
bank=#
```

Slika 2: Poruka nakon izvršavanja problematičnih transakcija na REPEATABLE READ nivou izolacije.

Druge transakcija je automatski prekinuta i njene izmene nisu upisane u bazu podataka.

Nakon završetka eksperimenta ako izvršimo upit za prikaz podataka tabele doctors, dobijamo sledeći rezultat:

id	name	on_call
1	Ana	f
2	Marko	t

(2 rows)

Slika 3: Rezultat transakcija na REPEATABLE READ nivou izolacije. Korektno stanje je očuvano.

Za razliku od prethodnog eksperimenta, poslovno pravilo sistema ostalo je očuvano. PostgreSQL je uspešno detektovao potencijalno opasnu strukturu zavisnosti između transakcija i prekinuo jednu od njih pre nego što je moglo doći do narušavanja serializabilnosti.

Dobijeni rezultat predstavlja potvrdu rada Serializable Snapshot Isolation mehanizma.

8. Zaključak

U okviru ovog rada analizirani su osnovni koncepti transakcija, ACID svojstva, anomalija konkurentnog izvršavanja i nivoa izolacije u sistemima baza podataka. Posebna pažnja posvećena je Multi-Version Concurrency Control (MVCC) arhitekturi i Serializable Snapshot Isolation (SSI) mehanizmu koji PostgreSQL koristi za implementaciju SERIALizable nivoa izolacije.

Kroz teorijsku analizu prikazano je na koji način Snapshot Isolation omogućava visok nivo konkurentnosti, ali istovremeno ne garantuje potpunu serializabilnost transakcija. Kao jedno od najznačajnijih ograničenja predstavljen je problem Write Skew anomalije.

Praktični deo rada pokazao je razliku između REPEATABLE READ i SERIALizable nivoa izolacije. Dok je u prvom slučaju bilo moguće narušiti poslovno pravilo sistema, korišćenjem SSI mehanizma PostgreSQL je uspešno detektovao konflikt između transakcija i sprečio nastanak nekonzistentnog stanja baze podataka.

Na osnovu sprovedene analize može se zaključiti da Serializable Snapshot Isolation predstavlja efikasno rešenje za obradu konkurentnih transakcija, omogućavajući visok nivo konzistentnosti podataka uz zadržavanje dobrih performansi i konkurentnosti sistema.

7. Literatura

- [1] PostgreSQL Global Development Group, *PostgreSQL Documentation – Transaction Isolation*. Dostupno na: <https://www.postgresql.org/docs/current/transaction-iso.html>
- [2] PostgreSQL Global Development Group, *PostgreSQL Documentation – Multi-Version Concurrency Control (MVCC)*. Dostupno na: <https://www.postgresql.org/docs/current/mvcc.html>
- [3] Cahill, M. J., Röhm, U., Fekete, A. D., *Serializable Isolation for Snapshot Databases*, ACM Transactions on Database Systems (TODS), Vol. 34, No. 4, 2009.
- [4] Kleppmann, M., *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
- [5] Silberschatz, A., Korth, H. F., Sudarshan, S., *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019.
- [6] Elmasri, R., Navathe, S. B., *Fundamentals of Database Systems*, 7th Edition, Pearson, 2016.
- [7] Bernstein, P. A., Hadzilacos, V., Goodman, N., *Concurrency Control and Recovery in Database Systems*, Addison-Wesley, 1987.
- [8] Fekete, A., O'Neil, E., O'Neil, P., *Making Snapshot Isolation Serializable*, ACM Transactions on Database Systems, 2005. Dostupno na: <https://dsf.berkeley.edu/cs286/papers/ssi-tods2005.pdf>
- [9] PostgreSQL Global Development Group, *PostgreSQL Documentation – Explicit Locking*. Dostupno na: <https://www.postgresql.org/docs/current/explicit-locking.html>